Recursive Program to Multiply Two Square Matrices

(Without Using Iterative Loops in the Multiplication Logic)

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

TABLE OF CONTENTS

1. Problem Statement
2. Objective
3. Introduction and Theory
4. Approach and Algorithm
5. Source Code
6. Sample Input and Output
7. Dry Run of the Recursion
8. Complexity Analysis
9. Conclusion

1. Problem Statement

Write a recursive program to multiply two square matrices. Iterative loops (for, while, do-while) must not be used in the multiplication logic.

2. Objective

- To understand how matrix multiplication works for two n × n matrices.
- To replace the three nested loops normally used for multiplication with recursive function calls.
- To practise writing recursive functions with a proper base case and recursive case.
- To implement the solution in the C programming language and verify it with test inputs.

3. Introduction and Theory

3.1 Square Matrix

A square matrix is a matrix that has the same number of rows and columns. A matrix of order n has n rows and n columns, so it holds n × n elements.

3.2 Matrix Multiplication

If A and B are two square matrices of order n, their product C = A × B is also a square matrix of order n. Each element of C is calculated as:

$$C[i][j] = A[i][0] \times B[0][j] + A[i][1] \times B[1][j] + \ldots + A[i][n-1] \times B[n-1][j]$$

In other words, C[i][j] is the sum of products of the elements of row i of A with the elements of column j of B.

3.3 Recursion

Recursion is a technique in which a function calls itself to solve a smaller part of the same problem. Every recursive function needs a base case, which stops the calls, and a recursive case, which moves the problem closer to the base case.

3.4 Replacing Loops with Recursion

The usual method uses three nested loops, over i (rows), j (columns) and k (the dot-product term). In this project the three loop variables i, j and k are passed as parameters to a single recursive function. Increasing k, then j, then i by recursive calls does exactly the work of the three loops.

4. Approach and Algorithm

The function multiply(A, B, C, n, i, j, k) works as follows:

Step 1. If i == n, all rows are finished. Return (base case).

Step 2. Else if j == n, the current row of C is complete. Call multiply with i + 1, j = 0, k = 0 and return.

Step 3. Else if k == n, the dot product for C[i][j] is complete. Call multiply with j + 1, k = 0 and return.

Step 4. Otherwise add one product term: C[i][j] = C[i][j] + A[i][k] × B[k][j].

Step 5. Call multiply again with k + 1 to process the next term.

Matrix C is first set to zero, because each element is built up by repeated addition. The functions for setting to zero, reading and printing the matrices are also written recursively, so that the whole program is free of loops.

Initial call: multiply(A, B, C, n, 0, 0, 0)

5. Source Code

Language: C File name: matrix_multiply.c

```
#include <stdio.h>

#define MAX 10

/* Recursively set every element of an n x n matrix to 0 */
void initMatrix(int M[MAX][MAX], int n, int i, int j)
{
    if (i == n)
        return;
    if (j == n)
    {
        initMatrix(M, n, i + 1, 0);
        return;
    }
    M[i][j] = 0;
    initMatrix(M, n, i, j + 1);
}

/* Recursively read an n x n matrix */
void readMatrix(int M[MAX][MAX], int n, int i, int j)
{
    if (i == n)
        return;
    if (j == n)
    {
        readMatrix(M, n, i + 1, 0);
        return;
    }
    scanf("%d", &M[i][j]);
    readMatrix(M, n, i, j + 1);
}

/* Recursively print an n x n matrix */
void printMatrix(int M[MAX][MAX], int n, int i, int j)
{
    if (i == n)
        return;
    if (j == n)
    {
        printf("\n");
        printMatrix(M, n, i + 1, 0);
        return;
    }
    printf("%6d", M[i][j]);
    printMatrix(M, n, i, j + 1);
}

/*
 * Recursive matrix multiplication: C = A x B
 *   i -> current row of A (and C)
 *   j -> current column of B (and C)
 *   k -> current position in the dot product
 */
void multiply(int A[MAX][MAX], int B[MAX][MAX], int C[MAX][MAX],
              int n, int i, int j, int k)
{
    if (i == n)                      /* all rows done */
        return;
```

```
    if (j == n)                      /* row finished, go to next row */
    {
        multiply(A, B, C, n, i + 1, 0, 0);
        return;
    }

    if (k == n)                      /* dot product finished, next column */
    {
        multiply(A, B, C, n, i, j + 1, 0);
        return;
    }

    C[i][j] += A[i][k] * B[k][j];    /* add one product term */
    multiply(A, B, C, n, i, j, k + 1);
}

int main()
{
    int A[MAX][MAX], B[MAX][MAX], C[MAX][MAX];
    int n;

    printf("Enter the order of the square matrices (1 to %d): ", MAX);
    scanf("%d", &n);

    if (n < 1 || n > MAX)
    {
        printf("Invalid order. Enter 1 to %d.\n", MAX);
        return 1;
    }

    printf("\nEnter elements of Matrix A (%d x %d):\n", n, n);
    readMatrix(A, n, 0, 0);

    printf("\nEnter elements of Matrix B (%d x %d):\n", n, n);
    readMatrix(B, n, 0, 0);

    initMatrix(C, n, 0, 0);
    multiply(A, B, C, n, 0, 0, 0);

    printf("\nMatrix A:\n");
    printMatrix(A, n, 0, 0);

    printf("\nMatrix B:\n");
    printMatrix(B, n, 0, 0);

    printf("\nResultant Matrix C = A x B:\n");
    printMatrix(C, n, 0, 0);

    return 0;
}
```

6. Sample Input and Output

Test Case 1: 3 × 3 matrices

```
Enter the order of the square matrices (1 to 10): 3

Enter elements of Matrix A (3 x 3):
1 2 3
4 5 6
7 8 9

Enter elements of Matrix B (3 x 3):
9 8 7
6 5 4
3 2 1

Matrix A:
     1     2     3
     4     5     6
     7     8     9

Matrix B:
     9     8     7
     6     5     4
     3     2     1

Resultant Matrix C = A x B:
    30    24    18
    84    69    54
   138   114    90
```

Test Case 2: 2 × 2 matrices

```
Enter the order of the square matrices (1 to 10): 2

Enter elements of Matrix A (2 x 2):
1 2
3 4

Enter elements of Matrix B (2 x 2):
5 6
7 8

Matrix A:
     1     2
     3     4

Matrix B:
     5     6
     7     8

Resultant Matrix C = A x B:
    19    22
    43    50
```

Both outputs were obtained by compiling and running the program above, and they match the results of manual calculation.

7. Dry Run of the Recursion

Consider A = [[1, 2], [3, 4]] and B = [[5, 6], [7, 8]], with n = 2. The table shows how the recursive calls update the result matrix C (starting from all zeros).

i	j	k	Operation	C[i][j] after
0	0	0	C[0][0] += 1 × 5	5
0	0	1	C[0][0] += 2 × 7	19
0	0	2	k == n, move to next column	19
0	1	0	C[0][1] += 1 × 6	6
0	1	1	C[0][1] += 2 × 8	22
0	1	2	k == n, j becomes 2, next row	22
1	0	0	C[1][0] += 3 × 5	15
1	0	1	C[1][0] += 4 × 7	43
1	1	0	C[1][1] += 3 × 6	18
1	1	1	C[1][1] += 4 × 8	50
2	–	–	i == n, base case reached, stop	Done

Final result: C = [[19, 22], [43, 50]].

8. Complexity Analysis

- Time complexity: $O(n^3)$. The function makes one call for every combination of i, j and k, which is n × n × n calls.
- Space complexity: $O(n^3)$ for the call stack, since the recursion depth grows up to about n^3 calls, plus $O(n^2)$ for the matrices.

Because of the deep recursion, the matrix size is limited to 10 × 10 in this program (MAX = 10). This keeps the stack usage safe on normal systems.

9. Conclusion

The program multiplies two square matrices using recursion only. The three loop variables i, j and k are carried as function parameters, and the base case i == n ends the process. The results for the test cases match manual calculation, so the program works correctly. Recursion is a good way to understand the structure of nested loops, although for large matrices the iterative method is more efficient because it uses less stack memory.